9.2 SOAL PRE-TEST

1. Jelaskan secara singkat tujuan penggunaan klausa JOIN pada basis data relasional dan apa kelemahannya jika digunakan pada tabel dengan jutaan baris data!

Jawaban:

Klausa JOIN digunakan untuk menggabungkan data dari dua tabel atau lebih berdasarkan kolom yang berelasi (primary key dan foreign key), sehingga data yang sudah dipisah pada proses normalisasi dapat ditampilkan kembali sebagai satu hasil. Pada SIM-PAK, misalnya, data aset, ruangan, dan laporan kerusakan disimpan di tabel terpisah (aset, ruangan_lokasi, laporan_kerusakan), dan JOIN dipakai untuk menampilkan laporan lengkap dengan nama alat dan lokasinya.

Kelemahannya, pada tabel dengan jutaan baris JOIN menjadi lambat dan boros sumber daya (CPU, memori, dan I/O disk), karena database harus mencocokkan baris antartabel. Beban makin berat jika kolom penghubungnya tidak diberi index atau jika JOIN dikombinasikan dengan fungsi agregasi dan GROUP BY, sehingga waktu respons sistem bertambah.

2. Apa yang dimaksud dengan redundansi data, dan mengapa dalam konsep "Denormalisasi" redundansi ini sengaja diciptakan?

Jawaban:

Redundansi data adalah kondisi ketika data yang sama atau data hasil turunan tersimpan berulang di lebih dari satu tempat dalam basis data. Pada desain yang ternormalisasi, redundansi sebisa mungkin dihindari agar data konsisten dan hemat penyimpanan.

Pada konsep denormalisasi, redundansi justru sengaja diciptakan untuk meningkatkan kecepatan pembacaan data. Dengan menyimpan nilai yang sudah dihitung (atribut turunan atau tabel rekapitulasi), sistem tidak perlu melakukan JOIN dan agregasi berulang setiap kali data diminta. Pada SIM-PAK, misalnya, dashboard rekapitulasi (KF-15) yang menampilkan frekuensi kerusakan alat akan dibuka berulang kali, sehingga lebih efisien jika jumlahnya disimpan langsung. Konsekuensinya, data harus dijaga agar tetap konsisten dan penyimpanan bertambah.

3. Berikan satu contoh konkret bagaimana sebuah atribut turunan dapat menghindari penggunaan fungsi agregasi (seperti `SUM`) dalam *query*!

Jawaban :

Contoh konkret: tabel aset ditambahkan kolom turunan jumlah_kerusakan yang menyimpan banyaknya laporan kerusakan tiap alat.

Sebelum (tanpa atribut turunan), jumlah kerusakan harus dihitung dengan JOIN dan fungsi agregasi COUNT setiap kali dibutuhkan:

```
SELECT a.kode_aset, a.nama_aset, COUNT(l.id_laporan)
AS jumlah_kerusakan

FROM aset a

LEFT JOIN laporan_kerusakan l ON a.kode_aset =
l.kode_aset

GROUP BY a.kode_aset, a.nama_aset;
```

Sesudah (dengan atribut turunan):

```
ALTER TABLE aset ADD COLUMN jumlah_kerusakan INT
DEFAULT 0;


UPDATE aset a

SET jumlah_kerusakan = (

  SELECT COUNT(*) FROM laporan_kerusakan l

  WHERE l.kode_aset = a.kode_aset

);

SELECT kode_aset, nama_aset, jumlah_kerusakan FROM
aset;
```

Dengan kolom *jumlah_kerusakan*, query cukup membaca satu tabel tanpa *JOIN,* tanpa *COUNT*, dan tanpa *GROUP BY,* tetapi hasilnya sama (misalnya PC-RPL-01 = 3 laporan kerusakan).

Atribut turunan juga dapat menggantikan fungsi agregasi lain seperti SUM. Contohnya, kolom total_biaya_perbaikan pada tabel aset menyimpan hasil SUM biaya perbaikan tiap alat, sehingga query cukup membaca satu tabel tanpa JOIN dan tanpa SUM. COUNT pada contoh di atas bekerja dengan prinsip yang sama.

Catatan: nilai jumlah_kerusakan hanya benar saat UPDATE dijalankan. Agar tetap konsisten, nilainya harus diperbarui setiap ada laporan kerusakan baru, misalnya dengan trigger AFTER INSERT pada tabel laporan_kerusakan atau dengan UPDATE berkala. Inilah konsekuensi denormalisasi: pembacaan data lebih cepat, tetapi proses penulisan dan penjagaan konsistensi menjadi lebih rumit.